DATA ENGINEERING & CLOUD · LESSON 14 OF 15

Linux and the Command Line for Data Work

Learn the terminal skills that data platforms depend on. Navigate the file system, read and search files, watch logs live, manage processes and permissions, and combine commands with pipes and redirection to answer operational questions quickly.

Gap-fill lesson | 2026 | Beginner | 120 min

By the end of this lesson you can

- Navigate and manage files and folders from the terminal
- Read, search and filter large text files and logs
- Use pipes and redirection to combine commands
- Check running processes and file permissions

Datasets used

`daily_sales.csv`

Why the terminal matters for data work

Most data platforms run on Linux servers, and the fastest way to answer "is the pipeline running?" or "what went wrong last night?" is the command line. Graphical tools are useful, but a terminal session on a server is the only interface many engineers ever get. Learning a small set of commands well saves hours during an incident.

CASE STUDY **The 2 a.m. log check**

The nightly load has not finished. The on-call engineer connects to the server, finds the log, filters for errors since midnight and sees a permission error on a shared folder. Ten minutes at the terminal fix what would otherwise take an hour of clicking through a web console.

Navigating the file system

Command	Purpose	Example
`pwd`	Show the current folder	`pwd`
`ls -lh`	List files with sizes	`ls -lh /data/landing`
`cd`	Change folder	`cd /data/landing`
`find`	Search for files by name, size or age	`find /data -name "*.csv" -mtime -1`
`du -sh`	Show the size of a folder	`du -sh /data/*`
`df -h`	Show free disk space	`df -h`

`find /data -name "*.csv" -mtime -1` lists CSV files changed in the last day. It is a common first step when a job says "file not found" but the file should be there.

Reading and searching files

Terminal

```
head -n 5 daily_sales.csv          # first five lines: check the header and format
tail -n 20 pipeline.log            # last twenty lines: what happened most recently
tail -f pipeline.log               # follow a log live as new lines are written
grep -n "ERROR" pipeline.log       # lines containing ERROR, with line numbers
grep -c "ERROR" pipeline.log       # count the matching lines
wc -l daily_sales.csv              # number of lines
```

`tail -f` is the command to keep open during a run. Press Ctrl+C to stop following. `grep -n` gives the line number, which you can then open with `sed -n '120,140p' file` to read the surrounding context.

Pipes and redirection

A **pipe** (`|`) sends the output of one command into another. **Redirection** (`>` and `>>`) sends output to a file. Together they answer operational questions in one line.

Terminal

```
# How many errors happened each hour in last night's log?
grep "ERROR" pipeline.log | cut -c1-13 | sort | uniq -c

# Save today's summary of rows per file, overwriting the old file
wc -l /data/landing/*.csv > row_counts.txt

# Append a timestamp line to a run history log
echo "$(date -Iseconds) load finished" >> run_history.log
```

Symbol	Meaning
`|`	Pass the output to the next command
`>`	Write the output to a file, replacing it
`>>`	Append the output to a file
`2>&1`	Send error messages to the same place as normal output

::::tip Use `>>` for logs, `>` for results Overwriting a log with `>` by mistake erases the history of every earlier run. Use `>>` for logs that must be kept, and `>` only for output you intend to replace. ::::

Processes

A **process** is a running program. Check what is running and stop what is stuck:

Command	Purpose
`ps aux | grep python`	Show Python processes and their owners
`top` or `htop`	Live view of processes, CPU and memory use
`kill PID`	Ask a process to stop (PID is the process number)
`kill -9 PID`	Force a process to stop; use only when `kill` fails

Stop a process gently first. A forced stop can leave a file half-written, which is why an idempotent job (see the reliability lesson) is safer to restart than one that appends.

Permissions

Every file has an owner, a group and permissions for read (r), write (w) and execute (x). `ls -l` shows them:

```
-rw-r-----  1 analyst  finance  4096 Nov 11 06:00 summary.csv
```

This file is readable and writable by `analyst`, readable by the `finance` group, and hidden from everyone else. The commands that change permissions are `chmod` (mode), `chown` (owner) and `chgrp` (group).

Need	Command	Example
Let the team read a report	`chmod g+r`	`chmod g+r summary.csv`
Make a script runnable	`chmod +x`	`chmod +x load.sh`
Give a file to another owner	`chown` (needs admin rights)	`sudo chown etl:etl summary.csv`

Grant the least permission that works. Do not set everything to world-readable to fix one error; find the group that needs access and add the file to it.

::::warn Read before you delete `rm -rf` removes files without asking and without a recycle bin. Before deleting, list the matches with `ls` or `find ... -print` and check the paths. Never run a delete command with a variable that might be empty. ::::

A checklist for a server incident

1. Confirm the time and the job: `date`, then the job's log with `tail -n 100`.
2. Search for the first error: `grep -n -m 1 "ERROR"`.
3. Check resources: `df -h` (disk) and `top` (CPU and memory).
4. Check the input: `ls -lh` and `head` on the expected file.
5. Check the process: `ps aux | grep <job>`.
6. Fix, rerun, and write a line in the run history with the cause.

INTERVIEW CORNER

What commands would you use to diagnose a failing batch job on a Linux server?

Check the job's log with `tail` and `grep` for the first error, confirm the input file exists and has the expected size and header with `ls` and `head`, check disk and memory with `df` and `top`, and see whether the process is still running with `ps`. Then fix the cause and rerun, keeping a record of what happened.

::::key What to remember - The terminal is the fastest way to inspect a server and its logs. - `tail -f`, `grep`, `head` and `wc` answer most first questions about a file. - Pipes combine small

commands; `>>` keeps history, `>` replaces it. - Stop processes gently first; prefer restartable, idempotent jobs. - Check permissions before changing them, and never delete without listing first.
::::

Quick quiz

5 questions · tap an answer to check it

1. Which command follows a log file as new lines are written?

A cat

B head

C tail -f

D wc -l

Answer: C. `tail -f` keeps the file open and shows new lines as they appear.

2. What does `command > file.txt` do?

A Appends to file.txt

B Writes the output to file.txt, replacing its contents

C Reads file.txt

D Deletes file.txt

Answer: B. `>` replaces the file; `>>` appends.

3. What does the pipe in `grep ERROR log.txt | wc -l` do?

A Counts the lines in the log

B Counts the lines containing ERROR

C Deletes the errors

D Prints the errors in red

Answer: B. The pipe passes the matching lines to `wc -l`, which counts them.

4. Why prefer `kill PID` before `kill -9 PID`?

A `kill -9` is slower

B A gentle stop lets the program finish cleanup, reducing the chance of half-written files

C `kill` deletes the process

D They are identical

Answer: B. Forced stops skip cleanup.

5. Which command shows free disk space?

A ls -l

B ps aux

C df -h

D grep -c

Answer: C. `df` reports disk usage by file system.

EXERCISE **Diagnose a sample log**

Create a sample log with twenty lines, including three ERROR lines and two WARNING lines (you can write it with a heredoc in the terminal). Use `grep`, `wc`, `tail` and a pipe to report the number of errors, the line number of the first error, and the last line of the log. Then write one command that appends a summary line to a separate history file.

Show solution (worked answer)

Hints: `grep -n -m 1 ERROR log.txt` gives the first error with its line number. `grep -c ERROR log.txt` counts the errors. `tail -n 1 log.txt` shows the last line. Use `>>` for the history file so earlier entries stay.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.